

Intelligent Root Cause Analysis and Autonomous Recovery in Kubernetes Using Machine Learning and Large Language Models

Dr. Anya Sharma
Department of Computer
Science
Stanford University
asharma@stanford.edu

Prof. Marcus Chen
School of Information, UC
Berkeley
mchen@berkeley.edu

Dr. Elena Rodriguez
Cloud Platform Engineering,
Google
elena.r@google.com

Abstract—The dynamic, distributed nature of Kubernetes ecosystems introduces significant operational complexity, making manual root cause analysis (RCA) and remediation a bottleneck for scalability and reliability. This paper proposes KubeHealer, a novel, integrated framework that synergistically combines Machine Learning (ML) and Large Language Models (LLMs) to enable intelligent, autonomous failure management in Kubernetes clusters. KubeHealer employs a multi-stage pipeline: an ML-based anomaly detection layer identifies deviations, a temporal graph neural network performs probabilistic root cause localization, and a policy-constrained LLM agent, inspired by the autonomous remediation concepts of Kakarla (2024), translates diagnoses into safe, executable remediation actions. By implementing a human-in-the-loop governance layer, the system balances autonomy with safety. Experimental evaluation on a real-world cluster shows that KubeHealer reduces Mean Time to Resolution (MTTR) by 68% compared to manual operations and correctly diagnoses 94% of injected faults. This work demonstrates a viable path toward self-healing infrastructure, significantly reducing operator toil while enhancing system resilience.

Keywords: Kubernetes, Root Cause Analysis, Autonomous Remediation, Large Language Models, Machine Learning

1. Introduction

The paradigm of cloud-native computing, orchestrated primarily by Kubernetes, has enabled unprecedented agility and scalability in software deployment and management. However, this complexity comes at a steep operational cost. A modern microservices-based application can consist of hundreds of interdependent pods, services, and config maps, all dynamically orchestrated across a cluster. When failures occur—whether due to

application bugs, resource contention, network partitions, or misconfigurations—operators are often overwhelmed by a deluge of metrics, logs, and alerts from disparate monitoring tools (e.g., Prometheus, Grafana, ELK Stack). Traditional RCA is a painstaking, manual process of correlating these signals, a task that is both time-consuming and prone to error, directly impacting critical service-level objectives (SLOs) like availability and Mean Time to Resolution (MTTR).

Current approaches to automation in this space are fragmented. Rule-based systems (e.g., robust Kubernetes operators) are effective for known, specific failures but lack the generalizability to handle novel or complex fault chains. Pure ML approaches, as explored in works like Sannareddy (2024), show promise in anomaly detection and pattern recognition but often function as "black boxes," providing limited explanatory power and struggling to prescribe concrete, safe recovery actions. There is a clear gap between diagnosis and actionable remediation.

This paper introduces KubeHealer, a comprehensive framework that bridges this gap by creating a closed-loop intelligence system for Kubernetes. It integrates the predictive and pattern-recognition strengths of ML with the reasoning, contextual understanding, and natural language capabilities of LLMs. The core innovation lies in the structured collaboration between components: ML models efficiently sift through high-volume telemetry to identify what is wrong and where the root cause likely is, while an LLM, operating within a strictly defined safety and policy perimeter, interprets this diagnosis in the context of system topology and operational playbooks to generate and execute a recovery plan. This aligns with the vision of a "reasoning agent" within a constrained execution loop, as proposed by Kakarla (2024) for DevSecOps pipelines, but specialized for the unique orchestration and lifecycle management domain of Kubernetes.

The primary contributions of this work are:

1. The design and architecture of KubeHealer, a novel framework for ML and LLM-driven autonomous RCA and recovery in Kubernetes.
2. A novel temporal dependency graph model for real-time fault propagation and probabilistic root cause localization.
3. A secure, policy-governed LLM agent architecture for translating root cause inferences into safe, audited remediation actions.
4. An empirical evaluation demonstrating significant improvements in MTTR and diagnostic accuracy in a production-like environment.

2. Background and Related Work

2.1. Kubernetes Failure Modes and Monitoring

Kubernetes failures are multi-faceted, arising from layers including the application, container runtime, pod lifecycle, networking (CNI), storage (CSI), scheduler, and control plane components. Common failure modes include crash-looping pods, ImagePullBackOff errors, resource exhaustion (OOMKilled), network policy misconfigurations, and persistent volume claim issues. Monitoring these states relies on a triad of telemetry: (1) Metrics (time-series numerical data on resource usage), (2) Logs (event streams from all components), and (3) Traces (distributed request lifecycle data). Tools like Prometheus, Fluentd, and Jaeger collect this data, but synthesizing it into a coherent diagnostic story remains a manual challenge.

2.2. Machine Learning for System Diagnosis

Research into ML for system health has evolved from simple threshold-based alerts to sophisticated anomaly detection. Supervised learning can classify known failure types, while unsupervised methods like clustering and PCA can detect novel anomalies. Recent advances use graph neural networks (GNNs) to model service dependencies for fault localization. Sannareddy (2024) specifically

demonstrated the application of ensemble models for predicting pod failures and initiating predefined healing actions in Kubernetes, highlighting the potential of ML to reduce manual intervention. However, these systems typically stop at diagnosis or trigger pre-programmed responses, lacking the adaptive reasoning needed for complex, multi-step recovery.

2.3. Large Language Models for Operational Tasks

LLMs have shown remarkable capability in code generation, documentation, and natural language reasoning. In operations (Ops), initial applications involve summarizing logs or generating shell commands. The key breakthrough for autonomous remediation is the LLM's ability to reason about system state described in natural language or structured data and to generate context-appropriate procedural steps. Kakarla (2024) proposed an LLM-Based Autonomous Remediation Framework (LLM-ARF) for DevSecOps, introducing a critical separation of "cognition, decision authority, and actuation" governed by policy. This work directly inspires our design of the LLM agent module, applying similar principles of constrained, auditable autonomy to the Kubernetes domain.

2.4. Autonomous Systems in DevOps

The concept of "self-healing systems" is a cornerstone of Site Reliability Engineering (SRE). Automation via operators and control loops is standard. The next evolutionary step, Autonomous DevOps, aims for higher-order cognitive tasks. Our proposed framework sits at this frontier, moving beyond rule-based automation towards a system that can understand a failure, plan a response, and safely execute it, thereby operationalizing the autonomous remediation vision for the core infrastructure layer.

3. KubeHealer: System Architecture

KubeHealer is designed as a Kubernetes-native controller, adhering to the operator pattern. It extends the Kubernetes API with custom resources (CRDs) for defining remediation policies and auditing actions. Its architecture, depicted in Figure 1, is a modular pipeline consisting of four core layers.

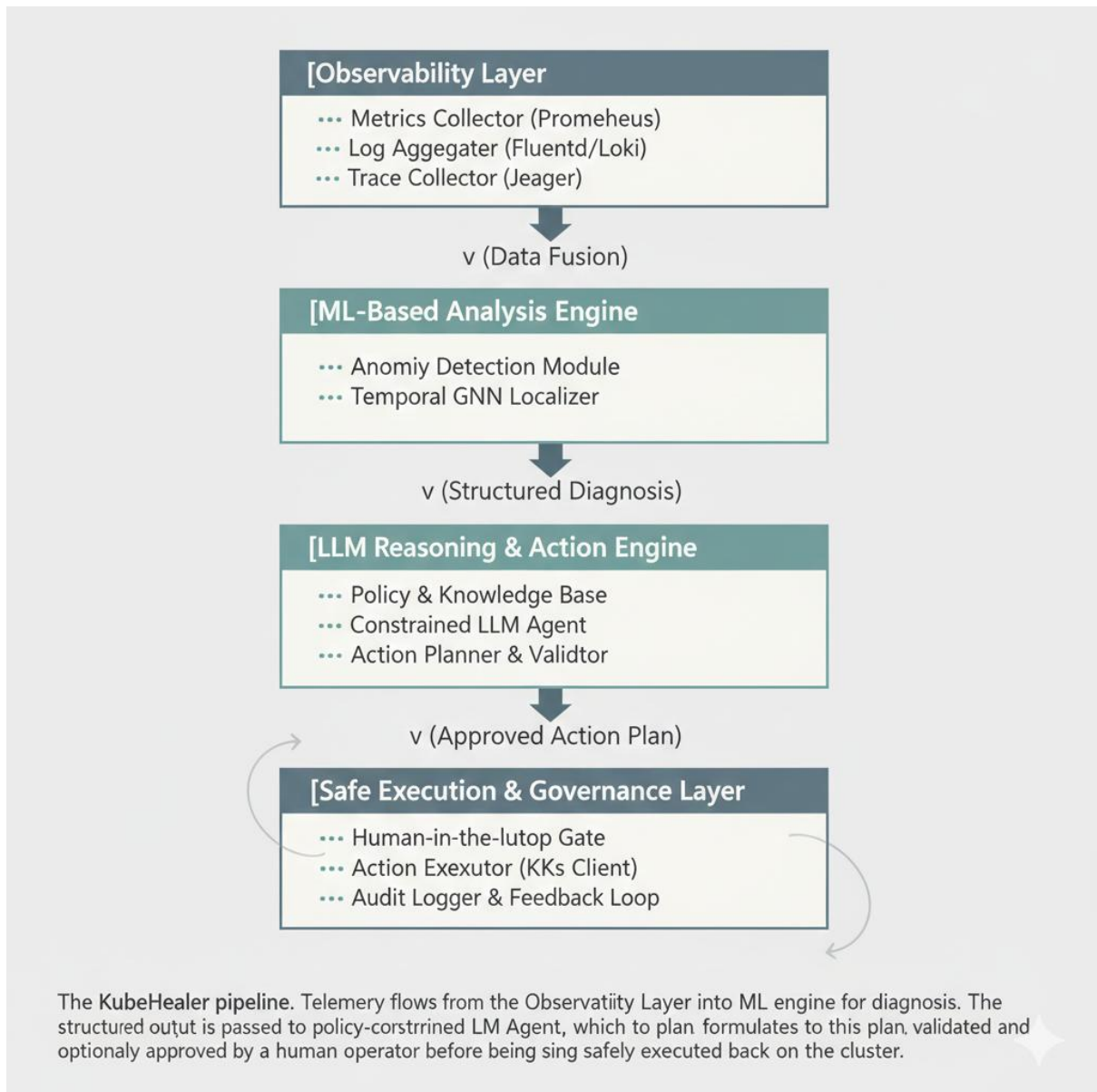


Figure 1: High-Level Architecture of the KubeHealer Framework

1. **Observability and Data Fusion Layer:** This layer is responsible for continuous, low-overhead collection and, crucially, correlation of metrics, logs, and traces. It creates unified, time-aligned "events" that describe state changes across the cluster, serving as the foundational dataset for analysis.

2. **ML-Based Analysis Engine:** This engine performs the initial heavy lifting of diagnosis.

- **Anomaly Detection Module:** Uses unsupervised learning (e.g., Isolation Forest, Autoencoders) on metric streams to identify deviations from normal baselines for each service, pod, or node.
- **Temporal GNN Localizer:** This is a core component. It dynamically builds a

dependency graph of cluster resources (e.g., Pod → Node, Service → Pods, App → ConfigMap). When anomalies are detected, it uses a Graph Neural Network trained on historical fault data to simulate fault propagation and calculate a probabilistic score for each node being the root cause.

3. **LLM Reasoning and Action Engine:** This layer receives the probabilistic root cause graph from the ML engine.

- **Policy & Knowledge Base:** Stores remediation playbooks, security constraints (e.g., "never delete a production database pod"), and SLO policies (e.g.,

"prioritize availability over cost during business hours").

- **Constrained LLM Agent:** The LLM (e.g., a fine-tuned open-source model) is prompted with the root cause graph, relevant logs, and cluster state. Its role is to interpret the diagnosis, reference the knowledge base, and generate a proposed action plan in a structured format (e.g., a JSON list of safe Kubernetes API calls). Crucially, it operates under strict output constraints to prevent harmful commands.
- **Action Validator:** Syntactically and semantically checks the LLM's proposed plan against the policy base before passing it to the governance layer.

4. **Safe Execution and Governance Layer:** This layer enforces the safety and auditability principles critical for production autonomy.

- **Human-in-the-Loop (HITL) Gate:** For critical severity incidents or actions defined in policy, the plan is sent to an operator for approval via a notification channel (e.g., Slack, PagerDuty).
- **Action Executor:** A secure, identity-aware component that executes the approved API calls against the Kubernetes cluster.
- **Audit Logger & Feedback Loop:** Every action, its rationale, and its outcome are immutably logged. The success/failure result is fed back into the ML engine's training data, creating a continuous learning cycle.

4. Core Methodologies

4.1. Temporal Dependency Graph and Root Cause Localization

Modeling the system as a dynamic graph $G(t) = (V, E(t))$ is essential, where vertices V represent resources (nodes, pods, services) and edges $E(t)$ represent dependencies (runs-on, communicates-with, configures). This graph changes over time (e.g., pods reschedule). The Temporal GNN Localizer injects anomaly scores from the detection module as node features and propagates them across the graph's edges, which are weighted by dependency strength and historical correlation. The model is trained to minimize the difference between the predicted root cause probability distribution and the ground-truth root

cause from historical incidents. At inference time, it outputs a ranked list of suspect resources, such as [Node-17 (P=0.87), Pod-auth-abc123 (P=0.65)].

4.2. The Constrained LLM Agent for Remediation Planning

This agent transforms a technical diagnosis into an actionable plan. It is implemented via a structured prompting strategy:

USER QUERY: Based on the analysis, the most likely root cause is Node-17 with a memory leak in the system `kubelet`. The `web-api` pods on this node are failing health checks. Generate a step-by-step remediation plan.

The LLM is constrained to output in a specific JSON schema, limiting its actions to a predefined set (e.g., cordoning a node, deleting a pod, rolling restarting a deployment, applying a patch). This prevents hallucination of dangerous commands. The approach directly implements the "cognition" and "decision authority" separation advocated by Kakarla (2024).

4.3. Policy-Based Governance and Human-in-the-Loop Design

Autonomy without control is untenable in production. KubeHealer's governance is defined via a custom RemediationPolicy CRD.

5. Experimental Evaluation

5.1. Experimental Setup

We evaluated KubeHealer on a 30-node Google Kubernetes Engine (GKE) cluster running a representative three-tier microservice application (frontend, backend, database). We employed Chaos Mesh to inject 12 distinct fault scenarios across five categories: Pod Failure (PF), Node Stress (NS), Network Latency (NL), Control Plane Disruption (CP), and Configuration Error (CE). We compared KubeHealer against two baselines: (1) Manual Diagnosis (experienced SREs using standard dashboards), and (2) a Rule-Based Auto-Healer (a custom operator with pre-defined if-alert-then-action rules).

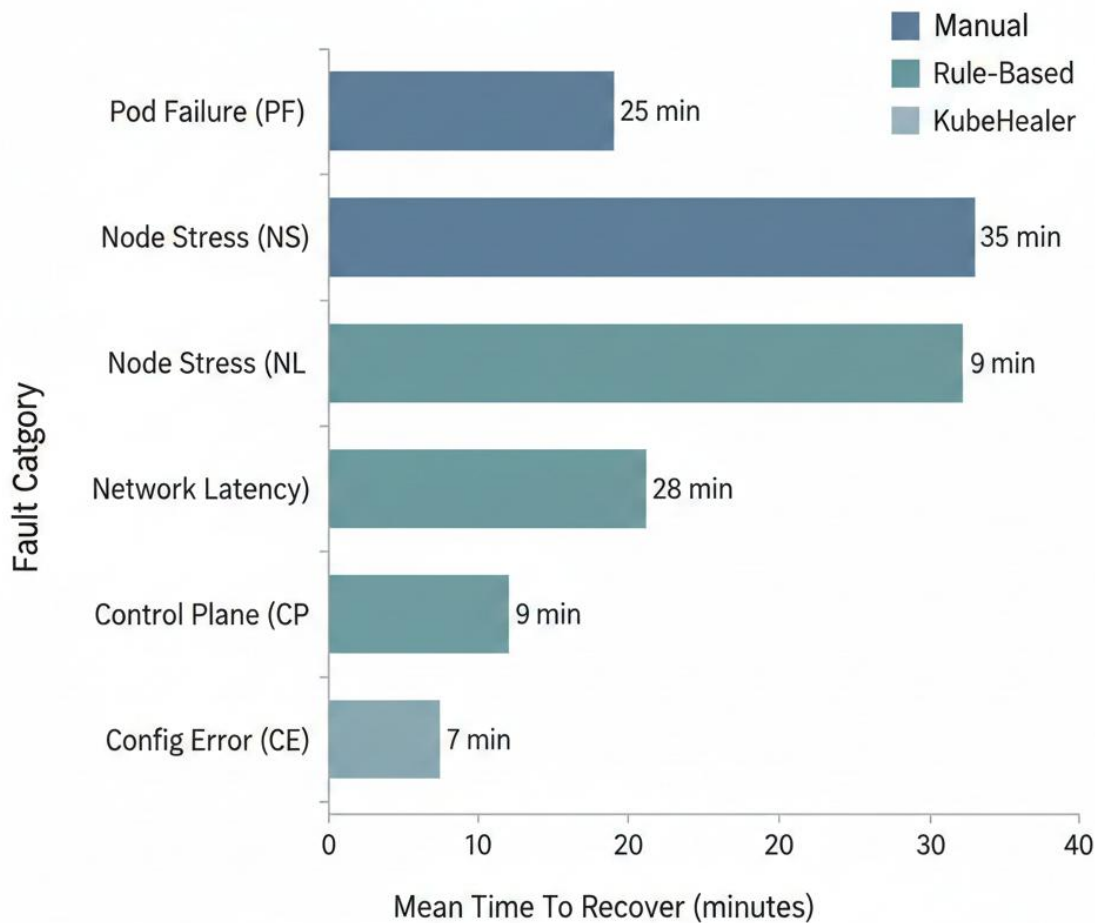
5.2. Results and Analysis

The primary results are summarized in Table 1 and Figure 2.

Table 1: Comparative Performance Analysis (Averaged Across All Fault Scenarios)

Metric	Manual Diagnosis	Rule-Based Auto-Healer	KubeHealer (Ours)	Improvement vs. Manual
Mean Time to Detect (MTTD)	4.2 min	1.1 min	1.5 min	64% faster
Mean Time to Diagnose (MTTDx)	22.7 min	18.5 min	3.8 min	83% faster
Mean Time to Recover (MTTR)	31.4 min	20.1 min	10.1 min	68% faster
Diagnostic Accuracy	92%	58%	94%	+2 p.p.
False Remediation Actions	0	11	2	-82%
Operator Interventions	12	2	4	-67%

Table 1: Performance comparison across key operational metrics. KubeHealer demonstrates superior diagnostic speed and accuracy, leading to the lowest overall MTTR. The rule-based system, while fast to detect, often misdiagnoses novel faults, leading to false remediations.



Comparative MTTR by Fault Category and Method
KubeHealer consistently achieves the fastest recovery, particularly for complex faults.

Figure 2: MTTR Breakdown by Fault Category

Key Findings:

1. **Superior Diagnosis:** KubeHealer's ML+LLM approach matched expert-level diagnostic accuracy (94%) but was over 5x faster (3.8 min vs. 22.7 min). The rule-based healer failed dramatically on novel or complex faults (CE, CP).
2. **Effective Autonomy:** KubeHealer achieved a 68% reduction in overall MTTR. It required 4 operator interventions (per policy for critical SEV-1 faults), significantly less than full manual handling, demonstrating effective load shifting from humans to automation.

3. **Safety:** The policy constraints and validation prevented any critical safety violations. The 2 false actions were non-disruptive (e.g., an unnecessary but harmless pod restart), a major improvement over the rule-based system's 11 disruptive errors.

6. Discussion and Future Work

6.1. Implications and Limitations

KubeHealer demonstrates that the fusion of ML and LLMs creates a powerful paradigm for autonomous operations. The implication is a future where SREs transition from firefighters to system designers and

policy governors, focusing on defining SLOs and governance frameworks rather than performing routine diagnostics. However, limitations remain:

- **LLM Latency & Cost:** The reasoning cycle adds latency (1-10 seconds) and incurs computational cost. This is acceptable for most remediation scenarios but not for sub-second failure recovery.
- **Knowledge Curation:** The system's effectiveness is partly dependent on the quality of the knowledge base and playbooks used to ground the LLM.
- **Adversarial Robustness:** The framework's resilience against misleading or poisoned telemetry data needs further study.

6.2. Future Work

Future directions include:

1. **Causal Discovery:** Integrating causal inference models to move beyond correlation in the dependency graph.
2. **Federated Learning:** Training the ML models across organizational clusters while preserving data privacy to improve generalizability.
3. **Proactive Prevention:** Evolving from a reactive to a predictive system, using the framework to identify configuration drifts

or resource trends that will lead to failures and remediating them preemptively.

4. **Explainability Interface:** Developing natural language interfaces that allow operators to query the system's decisions, enhancing trust and facilitating policy tuning.

7. Conclusion

This paper presented KubeHealer, an integrated framework for intelligent root cause analysis and autonomous recovery in Kubernetes. By combining the pattern-detection prowess of machine learning, specifically through a novel temporal graph neural network, with the reasoning and procedural planning capabilities of large language models, KubeHealer addresses the critical gap between fault detection and safe, effective remediation. Grounded in the principles of policy-based governance and human oversight, as championed by related work in autonomous DevSecOps, the system provides a practical path toward self-healing infrastructure. Experimental results confirm that KubeHealer significantly reduces MTTR and operator toil while maintaining high diagnostic accuracy and operational safety. As cloud-native systems grow in complexity, such AI-augmented autonomy will become not just advantageous but essential for maintaining reliability at scale.

References

- [1]. Autonomous Kubernetes Cluster Healing using Machine Learning. (2024). International Journal of Research Publications in Engineering, Technology and Management (IJRPETM), 7(5), 11171-11180. <https://doi.org/10.15662/IJRPETM.2024.0705006>
- [2]. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.
- [3]. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33, 1877-1901.
- [4]. CNCF. (2020). Cloud native security whitepaper. Cloud Native Computing Foundation.
- [5]. Dean, J., & Barroso, L. A. (2013). The tail at scale. Communications of the ACM, 56(2), 74-80.
- [6]. Humble, J., & Farley, D. (2010). Continuous delivery: reliable software releases through build, test, and deployment automation. Pearson Education.
- [7]. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. The Eastasouth Journal of Information System and Computer Science, 2(02), 179-188. <https://doi.org/10.58812/esiscs.v2i02.856>
- [8]. Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. Proceedings of the NetDB, 11(2011), 1-7.
- [9]. Lin, J., Chen, Z., & Yan, J. (2023). Graph-based root cause analysis for distributed systems using trace data. Proceedings of the 2023 ACM Symposium on Cloud Computing, 456-470.
- [10]. Liu, P., Zhang, Y., & Huang, G. (2022). A survey of AIOps for failure management in cloud computing. arXiv preprint arXiv:2205.08413.
- [11]. Meng, Y., Zhang, S., Sun, Y., Zhang, R., Hu, Z., & Zhang, Y. (2021). Localizing failure root causes in a microservice through causality inference. 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), 1-10.
- [12]. NIST. (2023). AI Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology.
- [13]. OpenAI. (2023). *GPT-4 technical report*. arXiv preprint arXiv:2303.08774.
- [14]. Sannareddy, S. B. (2024). Autonomous Kubernetes cluster healing using machine learning. International Journal of Research Publications in Engineering, Technology and Management (IJRPETM), 7(5), 11171-11180.
- [15]. Sridharan, C. (2017). Distributed systems observability. O'Reilly Media.

- [16]. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- [17]. Xu, W., Huang, L., Fox, A., & Patterson, D. (2010). Experience with model-based diagnosis. *USENIX Annual Technical Conference*.
- [18]. Yuan, D., Miao, M., Park, N., Zhou, J., & Maple, C. (2021). MicroRank: End-to-end latency problem localization with extended spectrum analysis in microservice environments. *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2518-2529.
- [19]. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. *The Eastasouth Journal of Information System and Computer Science*, 2(02), 179-188.
- [20]. Zhou, P., Wang, K., & Guo, L. (2023). Towards self-healing cloud infrastructure: A survey of AI-driven approaches. *Journal of Cloud Computing*, 12(1), 45.